DeepCodeGraph: A Language Model for Compile-Time Resource Optimization Using Masked Graph Autoencoders

Federico Cichetti(✉), Emanuele Parisi, Andrea Acquaviva, and Francesco Barchi

University of Bologna, 40126 Bologna, BO, Italy
federico.cichetti@studio.unibo.it,
{emanuele.parisi,andrea.acquaviva,francesco.barchi}@unibo.it

Abstract. Analysing source code using deep learning aids compile-time decisions affecting performance in embedded devices. We propose DeepCodeGraph, a general graph-based language model, which learns patterns to identify better compilation strategies, optimal hardware configurations and software transformations. DCG includes i) A large-scale dataset containing over 100k graphs. ii) A graph neural network to implement a graph-based language model. iii) A self-supervised pre-training framework leveraging Masked Graph Autoencoders. The performance of DCG is evaluated on two downstream tasks: heterogeneous device mapping and thread block size prediction. DCG outperforms previous graph-based state-of-the-art improving previous results by 3%.

Keywords: Masked graph autoencoder · Source code analysis · Compile-time decisions

1 Introduction

Contemporary computing systems frequently feature diverse hardware units, including CPUs, GPUs, or TPUs, making them *heterogeneous* devices. These devices can accelerate some classes of operations, but conventional compiler heuristics may struggle to adapt to the range of possible mapping options to fully leverage the potential of all platforms. Furthermore, optimal performance on a given compute unit can often only be achieved after rigorous exploration of device configuration parameters.

Statistical modelling and deep learning techniques applied to source code have been recently considered to address this complexity [1]. Notably, Language Models (LMs) and other Natural Language Processing (NLP) techniques are frequently deployed thanks to their ability to identify useful patterns for their designated task. In the domain of source code, these patterns may encapsulate knowledge regarding good software engineering practices, and its acquisition could facilitate the development of more intelligent, data-driven software engineering tools.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
A. Rapp et al. (Eds.): NLDB 2024, LNCS 14762, pp. 470–484, 2024.
https://doi.org/10.1007/978-3-031-70239-6_32

Techniques based on NLP handle source code files as language artefacts on the basis of a similarity between natural and programming languages [1]. However, source code inherently carries patterns related to the causal and temporal dependencies between data (e.g. variables and constants) and instructions, which can be challenging to identify in sequential formats. In fact, recent studies demonstrate that Large Language Models (LLMs), widely used to analyze natural language, can be ineffective when dealing with source code optimization and analysis [9,18]. It is also unclear whether existing training methodologies designed for LLMs of source code can effectively leverage the knowledge derived from this data type [22]. Graph-based representations can instead enhance understanding of high-level code concepts [15].

In recent years, the deep learning community has seen a transition from end-to-end models to transfer learning techniques. In this methodology, models undergo an initial pre-training phase on a significant dataset and are then fine-tuned for specific tasks of interest. This approach distinguishes the learning of a problem's solution from the extraction of inherent patterns in the data, allowing the general knowledge acquired during pre-training to be reused across multiple tasks. Following this trend, we aim to empirically validate the efficacy of graph-based learning by employing a self-supervised pre-training technique tailored for Graph Neural Networks (GNNs) and measuring the improvements that a pre-trained model introduces across two downstream tasks: i) mapping computational kernels to compute units (heterogeneous device mapping) and ii) determining the most efficient thread block size for a GPU kernel (thread block size prediction).

To this end, we develop "DeepCodeGraph" (DCG), a procedure for building a general language model for graph-based representations of source code. The key component of our methodology is an adaptation of the Masked Graph AutoEncoding (MGAE) framework. This technique involves masking a selection of random elements from an input graph and tasking the model with their reconstruction using the remaining information. While being a popular technique in other settings, to the best of our knowledge, prior research has not explored the utilisation of MGAEs as a pre-training method in the context of language models for source code analysis.

DeepCodeGraph outperforms prior graph-based techniques and pre-training procedures designed for code optimisation, improving state-of-the-art results on the aforementioned downstream tasks. In particular, we achieved an average accuracy of more than 87% on heterogeneous device mapping, improving the scores of previous graph-based models by 3.2%. Additionally, we obtained a 3% enhancement in thread size prediction, with a top-3 accuracy of over 19%.

The rest of this work is structured as follows: Sect. 2 reviews the main solutions found in the literature related to both source code analysis and MGAE techniques. Section 3 describes our DCG methodology. Section 4 examines pre-training and fine-tuning results, proposing different tests determined to evaluate the benefits introduced by our method. Lastly, Sect. 5 wraps up the work and highlights future directions.

2 Background and Related Works

The focus of this work is on the integration of probabilistic source code models into optimization tools, specifically addressing *heterogeneous device mapping* and *thread block size prediction*. These tasks optimize compile-time choices to enhance program efficiency. Heterogeneous device mapping identifies the most efficient device for executing a code segment, a crucial task in embedded systems with diverse hardware configurations. Thread block size prediction, instead, targets GPU-powered applications and aims to select the optimal thread size for a given kernel depending on the input matrix size. Both tasks require extensive experimentation and expertise due to elements such as dependencies on memory parallelization, cache usage and so on.

2.1 Source Code Techniques

Pioneering works for solving heterogeneous device mapping borrow techniques and models from the field of NLP. DeepTune [11] is a simple model composed of two LSTM layers producing a final dense semantic representation for a sequence of tokens extracted from tokenized OpenCL source code. Vavaroutsos et al. [29] improved the design of DeepTune by introducing a Bidirectional LSTM, including an initial convolutional layer and a final attention layer to emphasise and reweigh information before extracting a global representation.

More recently, several works have leveraged LLVM-IR source code as an alternative to high-level languages [2–4,8] showing impovements in results. LLVM-IR is a low-level, flexible and human-readable Intermediate Representation used for program analysis and optimisation within the LLVM compiler suite [23]. Relying on intermediate representations offers enhanced representational power and the ability to create language models independent of a specific source language. LLVM-IR explicitly includes memory-related operations (e.g., `alloca`, `load`, `store`), supports control flow analysis through ϕ-expressions and facilitates data flow analysis with Static Single Assignment (SSA) form [16].

Graph representations of source code have also been explored, as during compilation, source code transforms into different tree-based and graph-based forms. Abstract Syntax Trees (ASTs) depict syntax, while Control Flow Graphs (CFGs) and Data Flow Graphs (DFGs) represent code operations. A branch-free code segment ("basic block") can be represented as a DFG, while CFGs delineate the order of instructions and the conditions for the branches connecting the basic blocks. Novel source code graph representations can be created by combining these definitions, such as the Control and Data Flow Graph (CDFG) or the Code Property Graph [31].

Brauckmann et al. [8] integrate CDFGs with Graph Neural Networks (GNNs), specialised networks for graph data processing. Ben-Nun et al. [4] employed "conteXtual Flow Graphs" (XFGs) for learning an embedding space for LLVM-IR instructions (*inst2vec*) through the *skip-gram* approach.

More recently, ProGraML [15] has enabled parsing LLVM-IR files and transforming them into *directed multigraphs*. In these graphs, nodes correspond to

instructions, *variables*, and *constants* in code, while edges denote dependencies among them. These dependencies can be of three distinct types: *control* and *data*, reflecting control and data flow respectively, or *call*, symbolizing function calls and return-to-caller relations. Moreover, edges are enhanced with numerical values, providing positional information for distinguishing among parameter ordering or different CFG branches. Our work brings this exploration of source code representations for code analysis to the next step, proposing a framework for pre-training a general graph-based language model that can be fine-tuned to improve performance on several downstream tasks.

2.2 Masked Autoencoders on Graphs

Self-supervised learning (SSL) is a versatile concept applicable in various ways based on input data and tasks. Essentially, SSL enables the extraction of valuable knowledge directly from data, eliminating the need for costly manual labelling. This feature becomes especially attractive as the demand for data in training deep learning models continues to grow.

Contrastive Self-Supervised Learning (SSL) is widely used in graph data [30], emphasising the generation of multiple perspectives or "views" for each graph through carefully designed data augmentation. Models are then trained to maximise an agreement measure within the same graph views while minimising it between different graphs. However, criticism exists due to heavy reliance on the design of data augmentations and negative sampling techniques for pre-training. This dependency can be resource-intensive, time-consuming, and may not readily transfer across diverse graph types [20,24,30].

Contrarily, "Generative SSL" focuses on training models through a direct reconstruction of input graphs. This task is comparatively simpler and can be easily integrated into existing models. Encoder-decoder techniques are usually employed: the encoder network translates input graphs into a latent space, while the decoder reconstructs the masked elements of the original graph from the embedding. In order to avoid trivial mappings and enhance generality, the latent representation may have lower dimensionality or introduce noise to corrupt the input before encoding. In MGAEs, masking serves as the noise, hiding node and edge content by replacing embeddings with a "mask" token or removing connections.

Our approach, aligned with recent studies [20,21,24,27,28], leverages MGAEs as effective pre-training methods. We establish a novel connection between source code analysis and MGAEs, using them as a viable pre-training methodology for the specific data type of graph-based representations of source code. The learned LM then serves as an encoder in downstream tasks.

3 The DeepCodeGraph Methodology

In this section, we present DeepCodeGraph (DCG), the pipeline we designed for pre-training a graph-based language model using graph representations of

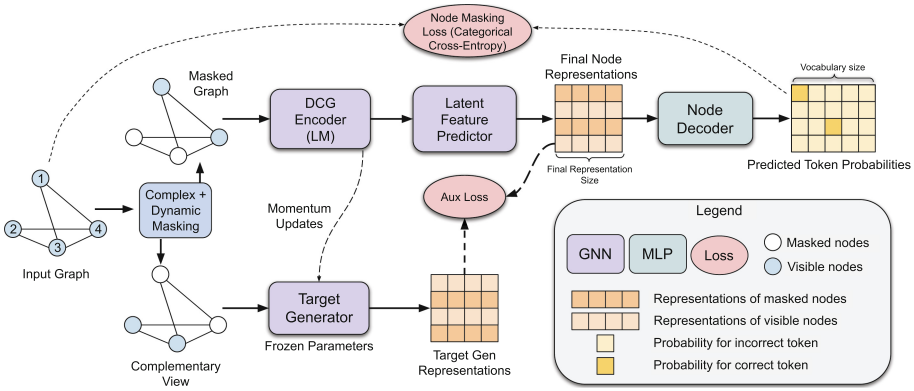


Fig. 1. DCG pre-training architecture.

source code and employing MGAEs as a pre-training framework. Additionally, we introduce the DCG dataset, a large collection of compiled OpenCL, C, and C++ source codes taken from different open-source benchmarks and projects. Lastly, we explain how our LM can be tailored for the downstream tasks outlined in Sect. 2.

3.1 Preliminaries

A graph representation can be expressed as a tuple $G = (V, E)$, where V is the set of nodes and E is the set of edges. Every node $v \in V$ represents an element from a dictionary of n possible tokens and can be mapped to a learnable feature vector h_v^0 by lookup in a fixed-size embedding table $D \in \mathbb{R}^{n \times d}$. Edges $e \in E$ can also contain information depending on the chosen graph format. For instance, edges in ProGraML [15] carry the *type* information (a categorical value specifying the kind of dependency among `DATA`, `CONTROL` or `CALL`) and the *position* information (an unbounded value representing operands order). Given an edge $e_{v,w}$ from node v to node w, these can be accessed by functions $\text{Type}(e_{v,w})$ and $\text{POS}(e_{v,w})$.

3.2 Model Description

The MGAE model implemented in this project is a modified version of RARE [28] introducing numerous optional components inspired by other works in the field. An overview of the whole architecture can be seen in Fig. 1. The model first randomly partitions the group of nodes into *masked* and *visible* nodes. Masking a node changes its initial features to those of a special `[MASK]` token. *Masked* nodes are selected with a probability p_{mask} that increases as training progresses (*dynamic masking*). There is also a lower probability $p_{rand} \leq p_{mask}$ that rather than a `[MASK]` token, the node is replaced by another random node from the vocabulary (*complex masking*). Both of these advanced masking techniques were inspired by HGMAE [27].

Subsequently, the masked graphs are input into our language model, which, during pre-training, serves as the encoder within the architecture. The language model is a GNN processing the input graphs through a local aggregation mechanism called *message passing*, repeated for a number of steps T. For the architecture of the GNN, we follow the design of [15]. At each step, $t \in [0, \ldots, T-1]$, three fundamental operations are executed: i) *Message emission*: A message for each pair of neighboring nodes (v, w) is generated based on current source features h_v^t, edge information and learnable matrix A:

$$msg_{v,w} = A_{\text{Type}(e_{v,w})}(h_v^t \odot \text{POS}(e_{v,w})) \tag{1}$$

ii) *Message aggregation*: Nodes receive messages from each incoming connection and aggregate them (e.g. by sum). iii) *Update*: Function U_t decides how to update the current set of features based on the current state and the aggregated message m_v^{t+1}.

$$h_v^{t+1} = U_t\left(h_v^t, m_v^{t+1}\right) \tag{2}$$

In this work, U_t is implemented as a Gated Recurrent Unit (GRU) cell [10]. At the end of message passing, node representations contain contextual information for their T-step neighbourhood. A final *readout operation* is used to obtain a global representation for the whole graph:

$$\hat{G} = \sum_{v \in V} \sigma\left(g_m(h_v^0, h_v^T)\right) \odot f_m(h_v^T) \tag{3}$$

After the LM, another small GNN (called *latent feature predictor* in [28]) further processes the representations. Concurrently, a third neural component with the same architecture as the LM, the *target generator*, computes representations for a complementary graph view (i.e. where masked nodes are visible and vice-versa). The *target generator* parameters ξ are not trained by backpropagation; instead, *momentum update* is used to manually update the weights from those of the LM (θ) every N training steps:

$$\xi \leftarrow \tau\xi + (1-\tau)\theta \tag{4}$$

The representations of the *target generator* are compared with those of the *latent feature predictor*, generating a self-supervised loss (mean squared error) that acts as regularisation for the primary training task of masked node reconstruction. Finally, a *decoder* (a simple multi-layer perceptron) is tasked to reconstruct the original node embeddings from the outputs of the *latent feature predictor*.

The original paper suggests utilizing a *reconstruction loss* based on the angle misalignment between the vectors of original and predicted node features [28]. However, we observed exceedingly poor pre-training results and low fine-tuning accuracy following this or similar approaches (e.g. [20,21]). We conjecture that by training both the embedding matrix D and the rest of the model, the loss minimization problem could lead to a naive solution, mapping all initial embeddings towards a singular point (e.g. 0) and pushing the decoder to consistently output values close to this cluster. To address this issue, we propose to re-frame the problem as a node token *classification*, similarly to the approach used in BERT

[17]. In this way, the model naturally learns discriminative embeddings in order to distinguish the correct option between all tokens in the vocabulary. To reflect the new task, the decoder was re-designed to produce probability distributions over the vocabulary, and the loss was changed to a standard cross-entropy, where the target is a one-hot encoding of the original token.

3.3 Codebase

To achieve generality, the LM must be exposed to a large collection of source code representations during the self-supervised training phase. The internet provides convenient access to an immense quantity of source code samples. However, as our representation relies on LLVM-IR, it is also necessary that the collected samples can be compiled using tools from the LLVM toolchain.

To this end, we introduce the DCG Dataset, a large, heterogeneous collection of compilable C/C++ and LLVM-IR code sourced from a wide range of open-source projects and publicly available benchmarks. This dataset encompasses various libraries for scientific computation, biological-oriented projects, and executable code from popular GitHub repositories. LLVM-IR source code files were obtained from pre-compiled collections (e.g., the CompilerGym project [14]) or were compiled from scratch using Clang [23]. The latter operation required, for each project, the adaptation of the compilation procedure. The ProGraML Python library [15] was then utilised to produce graph representations.

We discarded source code files that resulted in compilation errors and LLVM-IR files that took longer than 5 s to convert into graphs. Moreover, to ensure meaningful samples and stabilize the training process considering memory constraints, we excluded graphs with fewer than 5 or more than 3,000 nodes. These thresholds were selected on the basis of the distribution of unfiltered graph nodes, ensuring that not more than 10% of the graphs would be removed. Table 1 provides an overview of the source code distribution within the dataset.

3.4 Data Analysis

Vocabulary. The dataset was split into a training (80%), a validation (10%) and a test (10%) set. Particularly, POJ104 [25] was split per-problem, avoiding to include codes from the same problem in the different sets. The training set was used for data analysis and to collect a representative vocabulary of tokens for the graph transformation. While 99.5% of our training dataset can be covered with a dictionary of only 341 tokens, we further increased the size of the vocabulary up to 2,220 tokens for fair comparison with other vocabularies in literature (e.g. Cummins et al. use 2,230 tokens [13]). An additional "[UNK]" token is added for elements that are not found in the vocabulary.

Back Edges. In our analysis, we discovered that 6.1% of nodes lacked incoming connections. This scenario typically indicates variables and constants that are defined manually (i.e. not outputs of prior operations). Consequently, a significant portion of the graphs may never be influenced by the message-passing mechanism. To address this issue, we propose to introduce a fourth type of

Table 1. DCG Dataset description, indicating source project, number of source code, LLVM-IR and graph files.

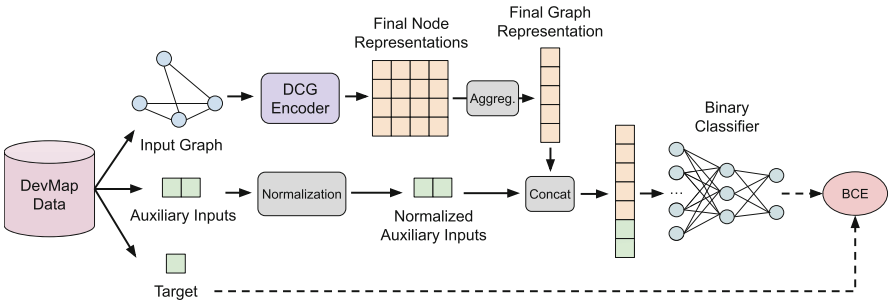
Name	C Files	Compiled	Graphs
Blas	300	300	216
bowtie2	57	57	25
bwa-mem	24	24	15
cBench	23	23	9
clang	711	711	66
CLGen	996	996	996
eigen	4,998	4,998	3,368
gemm_synth	3,700	3,700	3,072
Gromacs	1,249	1,205	828
JotaiBench	5,535	5,535	5,535
Linux	13,920	13,920	8,585
LLVM	21,371	21,371	17,598
MiBench	40	40	38
OpenCV	442	442	254
POJ104	49,816	49,815	49,804
stencil_synth	12,800	12,800	12,721
Tensorflow	1,985	1,985	683
Total	**117,967**	**117,922**	**103,813**

edge, `BACK`, connecting all targets of `DATA` dependencies back to their sources. `BACK` edges enable data nodes to be aware of the operations they are involved in, enriching the node representation and increasing communication within the graph with no additional parameters.

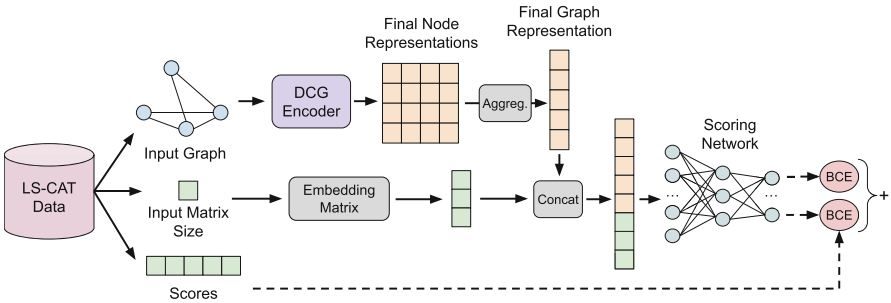
3.5 Models for Downstream Tasks

At the end of pre-training, we retain the parameters of the LM and remove the other components. Then, we adapt the LM to the downstream tasks by attaching some output layers learning to draw conclusions from the expressive graph representations.

The pipeline for heterogeneous device mapping is represented in Fig. 2a. We employ the DevMap dataset introduced in [11,12]. It contains a total of 256 OpenCL kernels paired with two dynamic values: the *OpenCL Work Group size* (affecting the amount of kernel parallelism) and the *Data* or *Payload size* (affecting the transfer time of data from host to device). Different values of workgroup/data size are used for some of the kernels, bringing the total number of samples to 680. These auxiliary inputs are normalized and processed into a tensor with fixed dimensionality that is concatenated to the graph representation output by the LM, following the procedure by [26]. Then, a MLP acts as a binary classifier, producing a probability distribution over the 2 allocation choices (CPU or GPU). The prediction is evaluated by binary cross-entropy loss against a one-hot encoded vector representing the fastest option.



(a) Process for the heterogeneous device mapping task.



(b) Process for the thread size prediction task.

Fig. 2. Input/output processing steps for the downstream tasks.

For thread size prediction we follow a similar approach (see Fig. 2b) and employ the recently introduced LS-CAT dataset [6]. The dataset focuses on auto-tuning in CUDA, specifically on the task of thread size prediction. Each CUDA kernel in the dataset is paired with an input matrix size, which is treated as an auxiliary input for the model and mapped to learnable features through an embedding matrix. The features are then concatenated to the final code graph representation. An MLP performs multi-label classification, assigning a score to each of the available thread sizes. Since the label for this task is a "performance score" for each of the thread sizes (see Sect. 4.3), the loss function is a sum of binary cross-entropy losses applied to the individual options.

4 Experimental Results

In this Section, we report the results of our experiments with the DCG pipeline, with a focus on the improvements that are obtained on the aforementioned downstream tasks.

4.1 Setup

Experiments. Depending on how the pre-trained components are handled, we define four tests that aim at exploring the quality of our methodology. The tests

Table 2. Configuration studies for fine-tuning.

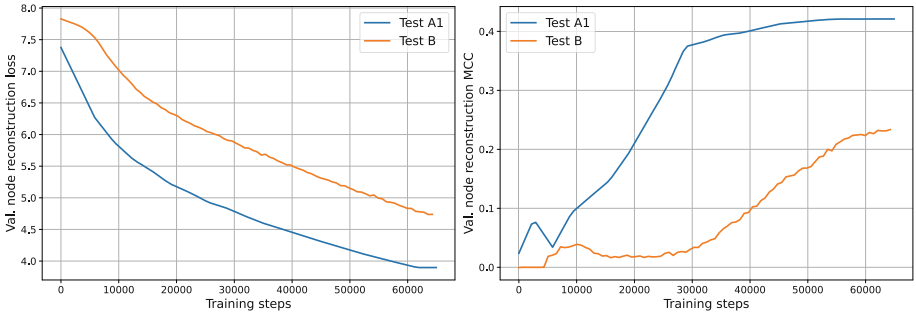
	Pre-Training			Model init in DevMap	
Test	Dataset	CodeRepr	Model	Embedding	Encoder
A1	DCG	ProGraML*	DCG	PreTrained	PreTrained
A2				PreTrained	Random
B	NCC	ProGraML*	DCG	PreTrained	PreTrained
C	NCC	XFG	inst2vec	PreTrained	n.a.

are summarised in Table 2 and can be described as follows: i) For *Test A*, we pre-train our DCG LM parameters and embedding matrix D by applying the self-supervised training framework of MGAE on the graphs of the DCG dataset During subsequent fine-tuning, we either load the set of pre-trained weights as the initial values of parameters (Test A1) or initialize them randomly (Test A2); however, the pre-trained embedding matrix D is always loaded. ii) In *Test B*, we apply the same methodology as Test A1, but pre-train the model on a best-effort reconstruction of the NCC dataset [4]. The vocabulary for this experiment was extracted from this dataset following our methodology and contains the same amount of tokens as the one described in Sect. 3.4. The objective of this test is to compare the NCC and DCG datasets: the latter is 4.5x larger, so the LM is expected to exhibit better loss and accuracy values. iii) In *Test C*, we use the inst2vec vocabulary and pre-trained embedding space [4] as external components. As model weights are not produced during pre-training, the fine-tuning phase always trains the model parameters from random initialization. This test aims to evaluate the quality of our DCG embeddings with respect to inst2vec.

Implementation. We implemented our models and data processing procedures with PyTorch Geometric [19]. For all experiments, we use $T = 6$ steps of message passing, a hidden dimensionality of 256 and an Adam optimizer. The initial embedding dimension $d = 200$ was chosen to have the same size of inst2vec embeddings [4]. We pre-train for 20 epochs with a mask rate increasing from 0.5 to 0.7 and a batch size of 32. Since the NCC dataset is approximately 4.5x times smaller, for test B the model is pre-trained for 90 epochs keeping all other hyperparameters the same to ensure a fair comparison. We then fine-tune on device mapping for 100 epochs and on thread size prediction for 40 epochs, with a learning rate of 2.5×10^{-4} reduced by a factor of 0.05 on loss plateaus. Device mapping employs a label smoothing factor of 0.05 for regularisation purposes.

4.2 Heterogeneous Device Mapping

The DevMap dataset reports, for each kernel, execution times on CPU and GPU for two separate heterogeneous devices. The devices share the same CPU but are equipped with different GPUs (NVIDIA and AMD). However, the version of the



(a) Node reconstruction loss. (b) Node reconstruction MCC.

Fig. 3. Validation loss and MCC score trends during training steps for pre-training configurations A1 and B.

dataset which is typically distributed contains undefined external functions and constants. To capture the full complexity of the kernels, the missing elements were reconstructed by hand using the source code from the original benchmarks as reference. This operation introduces greater complexity into the dataset but enables a more realistic analysis scenario.

The dataset is notoriously small, so *stratified K-fold cross-validation* (SKF) is employed for training and evaluating our models, averaging results across ten test folds. To ensure robustness against different random seeds, we report an average of ten experiments. As the dataset has a noticeable class imbalance [26], class weights are assigned to samples of the two classes in order to downweigh the loss for GPU samples (the most frequent). Table 4 shows the accuracy results of the different model configurations.

The results are all very similar and fall within the same confidence interval, except for test C, which exhibits a considerable drop in performance. In particular, in test A1, we reach outstanding results on the AMD subset of DevMap and obtain a slightly better average performance (.8713) than on test A2 (.8705) and test B (.8683).

We further study the difference between tests A1 and B. Purely in terms of pre-training results, we notice that pre-training on the DCG dataset allows the LM to produce more expressive representations, as can be seen by the lower decoding loss and higher node classification MCC score in Fig. 3. However, fine-tuning results show that the LM is able to quickly pick up on the downstream tasks even after pre-training on a less varied dataset. This does not make our dataset redundant, as test B still yields a lower performance with respect to A1 and A2. Note that the NCC dataset includes a partial overlap with the DevMap dataset, whereas the DCG dataset, by design, does not contain any code present in the DevMap dataset. Thus, these results support the hypothesis that the deployed MGAE approach has indeed provided the model with general and transferrable knowledge, giving it an edge with respect to other solutions.

Table 3. Comparison between prior works and our accuracy results on DevMap.

Model	Ref.	NVIDIA	AMD	Mean
DeepTune	[11]	.805	.814	.810
Inst2vec	[4]	.820	.828	.824
ProGraML	[15]	.800	.866	.833
DeepLLVM	[3]	.823	.853	.838
CDFG	[8]	.814	.864	.839
DeepTune-Exp	[29]	.815	.874	.844
DCG-A1 (Ours)		**.852**	**.890**	**.871**

Table 4. Results of our DevMap experiments.

	NVIDIA			AMD		
T	Mean		$CI_{95\%}$	Mean		$CI_{95\%}$
A1	.8524	±	.0020	**.8901**	±	**.0018**
A2	**.8553**	±	**.0042**	.8856	±	.0018
B	.8500	±	.0028	.8865	±	.0042
C	.8146	±	.0034	.8591	±	.0019

In Table 3, we compare our methodology with prior approaches in terms of average test accuracy. DCG improves upon all previous graph-based source code analysis models, setting a new state-of-the-art in heterogeneous device mapping.

4.3 Autotuning (LS-CAT)

The LS-CAT dataset [6] comprises over 20K CUDA kernels collected from larger GitHub projects. The kernels have been isolated and processed in order to allow their compilation and execution as stand-alone functions. The authors provide average execution times on a Tesla T4 GPU. Each kernel is executed with 7 different input matrix sizes, serving as auxiliary input, and 20 thread block sizes, constituting the decision space. Since non-optimal choices are still able to achieve improvements with respect to the average scenario, [6] proposes to assign penalties to the model based on a *performance* metric measuring the distance between the predicted and the optimal choice in a 0–1 range:

$$p_{(k,i,b)} = \frac{\min_x T_{(k,i,x)}}{T_{(k,i,b)}} \tag{5}$$

where T are times, k, i and b are respectively kernels, input sizes and block sizes.

While useful for evaluation, this metric tends to have high values for almost all choices. For this reason, performance scores are further processed with a transformation, aimed at raising the entropy in predicted probability distributions:

$$m_{(k,i,b)} = \frac{e^{Cp_{(k,i,b)}}}{\sum_x e^{Cp_{(k,i,x)}}} \qquad l_{(k,i,b)} = \frac{m_{(k,i,b)}}{\max_x m_{(k,i,b)}} \tag{6}$$

where C is a constant set to 12 and l represents the actual training labels. Finally, the loss function is a sum of binary cross-entropy terms for each of the options evaluating the alignment between the performance labels and the outputs.

One-tenth of the dataset is held out as a test set, one-tenth as a validation set, while the remaining 80% is used as a training set. The model that performs best on its validation fold is evaluated against the test set. To the best of our

Table 5. Test metrics for thread size prediction on LS-CAT.

Name	Top-1 Acc.	Top-3 Acc.	Perf.
Random Selection	0.050	0.150	0.92
Fixed Selection (1D, 1024)	0.056	0.160	0.93
fastText+LSTM	n.a.	n.a.	0.94
DCG-A1 (Ours)	0.069	0.191	0.93

knowledge, only another work by Bjertnes et al. [5] currently reports results on LS-CAT, although they only provide the performance metric of their model. Their best approach is token-based, using fastText [7] embeddings for LLVM-IR tokens and a simple LSTM-based LM. We compare our best configuration (A1) against theirs, reporting top-1, top-3 and performance results averaging over five different experiments with varying random seeds in Table 5. We also report the same metrics for random and fixed selection (i.e. always choosing the largest thread block size). We achieved comparable state-of-the-art results on the task, with a performance of 0.93 and a top-3 accuracy of over 19%.

5 Conclusions

Modern computing systems pose challenges for traditional compiler heuristics due to increasing complexity and diverse hardware specialisation. Recent studies leverage machine learning, particularly neural networks, for compile-time decisions. This paper introduces a novel language model for source code analysis, employing a Graph Neural Network pre-trained with a Masked Graph Autoencoding (MGAE) technique. Our approach, called DeepCodeGraph (DCG), utilizes an expanded ProGraML format for code processing. We adapt and extend RARE [28], a MGAE technique, enhancing its suitability for source code. DCG has been evaluated on two tasks: heterogeneous device mapping and thread size prediction. We observed a 3% improvement in DevMap test accuracy when comparing DCG to similar techniques. We also report a 3% enhancement in LS-CAT top-3 accuracy with respect to a fixed selection policy. Additional contributions include expanding the dataset for heterogeneous device mapping through the re-introduction of external elements, and creating the DCG dataset, a collection of approximately 100k graph representations of source code which is 4.5x larger than similar datasets in literature (i.e. NCC [4]).

References

1. Allamanis, M., et al.: A survey of machine learning for big code and naturalness. ACM Comput. Surv. **51**(4), 81:1–81:37 (2018). https://doi.org/10.1145/3212695
2. Barchi, F., et al.: Code mapping in heterogeneous platforms using deep learning and llvm-ir. In: Proceedings of the 56th Annual Design Automation Conference 2019. Dac '19. Association for Computing Machinery, New York (2019). https://doi.org/10.1145/3316781.3317789